

5. Modellierung der Daten

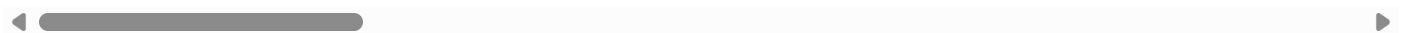
</> Code ▾

► Code

Train: (45920, 28)

	name	geber	art	jahr	anschrift	politikbereich	zweck	betrag	empfänger
id									
33655	Cashmere Radio e. V.	Senatsverwaltung für Kultur und Gesellschaftli...	Projektförderung	2024	Frankfurter Allee 7, 10247 Berlin	Kultur	signal2noise ? Art, Aesthetics And Social Prac...	120000	vr_0
105910	Merantix Labs GmbH	Senatsverwaltung für Wirtschaft, Energie und B...	Projektförderung	2022	Max-Urich-Straße 3, 13355 Berlin	Wirtschaft	Errichtung einer Betriebsstätte	5481380	hrb_0
67465	Georg-Kolbe-Stiftung	Senatsverwaltung für Kultur und Europa	Projektförderung	2020	Sensburger Allee 25, 14055 Berlin	Kultur	Der absolute Tanz- Festival sculpture	70000	spr_0
159301	Verschiedene Gesellschaften bürgerlichen Rechts	Senatsverwaltung für Inneres, Digitalisierung ...	Projektförderung	2023	'---'	Sport	Kosten für die Beschäftigung von Übungsleitern	1380	NaM
19495	Berliner Rugby-Club	Senatsverwaltung für Inneres und Sport	Projektförderung	2020	Scharfestr. 12, 14169 Berlin	Sport	anteilige Finanzierung des Spielbetriebes der ...	9000	vr_0

5 rows × 28 columns



5.1. Vorbereitung der Modellierung

Die Vorverarbeitung wird als scikit-learn `Pipeline` aufgebaut, weil dadurch alle lernenden Schritte innerhalb der Cross-Validation nur auf den jeweiligen Trainingsdaten angepasst werden. Das reduziert das Risiko von Data Leakage.

5.1.1. Zielvariable und Eingabemerkmale trennen

Zuerst werden Zielvariable und Eingabemerkmale getrennt. Als Zielvariable wird `politikbereich` verwendet. Für die Eingabemerkmale werden die bereits bereinigten und standardisierten Textspalten, die numerischen Spalte `betrag` sowie kategoriale Spalten `art` und `jahr` genutzt. Die Empfänger-ID wird hier bewusst nicht als direktes Modellmerkmal verwendet, weil sie stark organisationsspezifisch ist und das Modell sonst eher bekannte Empfänger memorisieren könnte.

► Code

	feature	missing_values
0	name_standardised	0
1	geber_standardised	0
2	anschrift_standardised	0
3	zweck_standardised	1
4	betrag	0
5	art_standardised	0
6	jahr	0

5.1.2. Textspalten vorbereiten

Die Textspalten können nicht direkt von klassischen scikit-learn-Modellen verarbeitet werden. Deshalb werden sie in TF-IDF-Merkmale umgewandelt. TF-IDF ist für diese Aufgabe geeignet, weil nicht nur das Vorkommen eines Wortes zählt, sondern auch seine Spezifität innerhalb des gesamten Korpus. Sehr seltene Terme werden mit `min_df=2` entfernt, da sie häufig Tippfehler oder Einzelfälle darstellen. Sehr häufige Terme werden mit `max_df=0.90` reduziert, weil sie meist wenig trennscharf sind. Mit `gram_range=(1, 2)` werden zusätzlich Wortpaare berücksichtigt, z. B. fachlich relevante Ausdrücke wie „soziale stadt“.

TF: Term Frequency IDF: Inverse Document Frequency

► Code

5.1.3. Numerische Spalten vorbereiten

Die numerischen Merkmale werden getrennt von den Textspalten verarbeitet. Fehlende Werte werden mit dem Median ersetzt, weil der Median robuster gegenüber extremen Förderbeträgen ist als der Mittelwert. Anschließend werden die numerischen Werte standardisiert. Das ist besonders für lineare Modelle wichtig, weil Merkmale mit großer Skala sonst einen unverhältnismäßig starken Einfluss bekommen können.

► Code

5.1.3. Kategorische Spalten vorbereiten

Für die kategorialen Merkmale werden in diesem Notebook `art_standardised` und `jahr` verwendet. Die Spalte `art_standardised` beschreibt die Art der Förderung und besitzt keine natürliche numerische Reihenfolge. Auch `jahr` wird in diesem Modellierungsansatz nicht als kontinuierliche Zahl behandelt, sondern als diskrete Kategorie. Dadurch kann das Modell jahresspezifische Unterschiede berücksichtigen, ohne eine lineare zeitliche Entwicklung zwischen den Jahren vorauszusetzen.

Die Verarbeitung erfolgt mit einer scikit-learn `Pipeline`. Zunächst werden fehlende Werte mit `SimpleImputer(strategy="most_frequent")` durch die häufigste Kategorie ersetzt. Danach werden die Kategorien mit `OneHotEncoder` in binäre Merkmale umgewandelt. Für jede Ausprägung entsteht dadurch eine eigene Indikatorspalte. So kann das Modell kategoriale Informationen nutzen, ohne dass eine künstliche Ordnung zwischen den Kategorien entsteht.

Mit `handle_unknown="ignore"` bleibt die Pipeline außerdem stabil, falls in späteren Daten eine Kategorie auftritt, die im Training nicht vorhanden war. Diese unbekannte Kategorie führt dann nicht zu einem Fehler, sondern wird in der One-Hot-Repräsentation ignoriert.

► Code

5.1.4. Spaltenspezifische Vorverarbeitung zusammenführen

Da Text- und numerische Spalten unterschiedliche Verarbeitung benötigen, werden sie mit einem `ColumnTransformer` zusammengeführt. Jede Textspalte erhält eine eigene TF-IDF-Repräsentation, damit der Kontext der jeweiligen Spalte erhalten bleibt. Der Förderzweck, der Geber und der Empfängername werden dadurch nicht zu einem einzigen Textblock vermischt, sondern als getrennte Informationsquellen behandelt.

► Code

Preprocessor definiert für 7 Eingabespalten.

5.2. Evaluationsrahmen vorbereiten

Wegen der Klassenunverteilung wird die spätere Modellbewertung nicht nur über Accuracy erfolgen. Accuracy zeigt zwar den Anteil aller korrekt vorhergesagten Fälle, kann bei unausgewogenen Klassen jedoch stark durch große Klassen dominiert werden. Deshalb werden zusätzliche Metriken verwendet, die die Leistung über alle Politikbereiche hinweg differenzierter bewerten.

Der Macro-F1-Score wird als Hauptmetrik verwendet, da er jede Klasse gleich stark gewichtet und dadurch auch Probleme bei kleinen Klassen sichtbar macht. Der Weighted-F1-Score ergänzt diese Sicht, indem er die tatsächliche Klassenhäufigkeit berücksichtigt. Zusätzlich wird Balanced Accuracy verwendet, weil sie die durchschnittliche Erkennungsrate über alle Klassen hinweg misst.

Metriken pro Klasse und die Konfusionsmatrix werden nicht direkt im Cross-Validation-Scoring berechnet, sondern später bei der detaillierten Modellbewertung ergänzt. Sie helfen dabei zu erkennen, welche Politikbereiche besonders häufig falsch klassifiziert werden und welche Klassen vom Modell kaum erkannt werden.

Metrik	Zweck
Macro-F1	Gleichwertige Bewertung aller Klassen
Weighted-F1	Bewertung entsprechend der Klassenhäufigkeit
Balanced Accuracy	Durchschnittliche Erkennungsrate aller Klassen
Accuracy	Anteil aller korrekten Vorhersagen
Metriken pro Klasse	Erkennung problematischer Klassen
Konfusionsmatrix	Darstellung typischer Verwechslungen

► Code

Vorbereitete CV-Folds: 4

► Code

Hauptmetrik: macro_f1

Verwendete Metriken: ['macro_f1', 'weighted_f1', 'balanced_accuracy', 'accuracy']

► Code

5.3. Geeignete Modellhypothesen für den Politikbereich-Klassifikator

Die Aufgabe ist eine mehrklassige Klassifikation des **politikbereich**. Die Eingabedaten bestehen aus mehreren bereinigten Textspalten, kategorialen Merkmalen und numerischen Informationen. Gleichzeitig ist die Zielvariable deutlich unausgewogen. Deshalb werden zunächst Modelle gewählt, die einerseits mit hochdimensionalen Textmerkmalen umgehen können und andererseits eine robuste, nachvollziehbare Vergleichsbasis liefern.

Die Modellhypothesen werden in drei Gruppen getrennt. **Klassische lineare Modelle** werden zuerst umgesetzt, weil sie schnell, reproduzierbar und gut mit Cross-Validation kombinierbar sind. **Tabellarische Modelle für gemischte Merkmale** werden zusätzlich geprüft, weil der Datensatz neben Text auch kategoriale und numerische Informationen enthält. **NLP- bzw. Deep-Learning-Modelle** werden als spätere Erweiterung betrachtet, da sie deutlich mehr Rechenzeit, zusätzliche Trainingslogik und häufig eine separate Validierungsstrategie benötigen.

Klassische lineare Modelle

Modellkandidat	Begründung für diesen Datensatz	Rolle im Projekt
Dummy Classifier	Sagt nur die häufigste Klasse voraus und nutzt keine Eingabemerkmale. Dadurch entsteht ein einfacher Mindestvergleichswert für alle echten Modelle.	Referenzmodell
Linear SVM / LinearSVC mit TF-IDF	Textdaten mit TF-IDF erzeugen sehr viele sparse Merkmale. Lineare SVMs gelten als besonders geeignet für hochdimensionale Textklassifikationsprobleme. Es werden Varianten mit und ohne Klassengewichtung sowie unterschiedlicher Regularisierung getestet.	Starker klassischer Benchmark
SGDClassifier mit Hinge Loss	Approximiert eine lineare SVM mit stochastischem	Effiziente SVM-Alternative

Modellkandidat	Begründung für diesen Datensatz	Rolle im Projekt
	Gradientenverfahren. Diese Variante kann bei großen sparse Textdaten rechnerisch schneller sein als LinearSVC .	
SGDClassifier mit Log Loss	Verwendet stochastisches Gradientenverfahren für eine logistische Regression. Damit dient das Modell als schnelle, skalierbare Variante der logistischen Regression.	Schnelle logistische Baseline
Logistische Regression mit TF-IDF	Ebenfalls ein starker linearer Textklassifikator, liefert gut interpretierbare Koeffizienten und unterstützt Klassengewichtung.	Interpretierbare lineare Baseline
Complement Naive Bayes	Sehr schneller Textklassifikator, der speziell als Verbesserung von Naive Bayes für unausgewogene Textdaten vorgeschlagen wurde. Da numerische und kategoriale Zusatzmerkmale nur eingeschränkt zu Naive Bayes passen, wird dieses Modell als textbasierte Zusatzbaseline betrachtet.	Schnelle einfache Textbaseline

Tabellarische Modelle für gemischte Merkmale

Modellkandidat	Begründung für diesen Datensatz	Rolle im Projekt
CatBoost	CatBoost kann kategoriale Merkmale direkt verarbeiten und unterstützt zusätzlich Textmerkmale. Dadurch eignet sich das Modell als Vergleich zu den linearen TF-IDF-Modellen, weil name , geber , anschrift , zweck , art , jahr und betrag gemeinsam in einer Modellfamilie genutzt werden können.	Nicht-lineare Mixed-Feature-Baseline

Modellkandidat	Begründung für diesen Datensatz	Rolle im Projekt
fastText	Effizientes Textklassifikationsmodell auf Basis von Wort- und n-Gramm-Repräsentationen. Da es nicht direkt in die bestehende scikit-learn-Pipeline passt, wird es als separater NLP-Vergleich betrachtet.	Effiziente NLP-Alternative
German BERT / multilingual BERT	Transformer-Modelle können semantischen Kontext besser erfassen als reine TF-IDF-Modelle. Sie sind jedoch rechenintensiver, benötigen Tokenisierung und Fine-Tuning und sind weniger direkt interpretierbar.	Erweiterte Deep-Learning-Hypothese bei ausreichender Zeit

Für dieses Projekt ist es sinnvoll, zuerst mit den klassischen TF-IDF-basierten Modellen zu beginnen. Diese Modelle liefern eine transparente Vergleichsbasis und zeigen, wie weit man mit gut kontrollierbaren linearen Verfahren kommt. Ergänzend wird CatBoost als nicht-lineares Modell für gemischte Merkmale geprüft. fastText und BERT werden nicht als erste Baseline, sondern als spätere NLP-Erweiterung eingeordnet.

Die SVM-Varianten prüfen den Einfluss von Regularisierung und Klassengewichtung. Der `SGDClassifier(loss="hinge")` wird als rechnerisch effiziente SVM-Alternative aufgenommen. Der `SGDClassifier(loss="log_loss")` ergänzt dies als schnelle logistische Variante. CatBoost prüft dagegen, ob die gemeinsame Nutzung von Text-, Kategorie- und Zahlenmerkmalen zusätzliche nicht-lineare Muster sichtbar macht.

Wichtig ist, dass der Modellvergleich nicht primär über Accuracy erfolgt. Wegen der Klassenunverteilung wird der **Macro-F1-Score** als Hauptmetrik verwendet. Zusätzlich werden Weighted-F1, Balanced Accuracy, Metriken pro Klasse, Konfusionsmatrix sowie die Trainingszeit betrachtet.

Literaturhinweise:

- Joachims, T. (1998): *Text Categorization with Support Vector Machines*. https://www.cs.cornell.edu/people/tj/publications/joachims_98a.pdf
- Fan, R.-E. et al. (2008): *LIBLINEAR: A Library for Large Linear Classification*. <https://www.jmlr.org/papers/v9/fan08a.html>
- scikit-learn Dokumentation: `SGDClassifier`. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDClassifier.html
- Prokhorenkova, L. et al. (2018): *CatBoost: unbiased boosting with categorical features*. <https://arxiv.org/abs/1706.09516>
- Rennie, J. D. M. et al. (2003): *Tackling the Poor Assumptions of Naive Bayes Text Classifiers*. <https://people.csail.mit.edu/jrennie/papers/icml03-nb.pdf>
- Joulin, A. et al. (2017): *Bag of Tricks for Efficient Text Classification*. <https://aclanthology.org/E17-2068/>

- Devlin, J. et al. (2019): *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. <https://aclanthology.org/N19-1423/>
- Wahba, Y. et al. (2022): *A Comparison of SVM against Pre-trained Language Models for Text Classification Tasks*. <https://arxiv.org/abs/2211.02563>

5.4. Definition der Modelle

In diesem Schritt werden die ersten Modellkandidaten definiert, aber noch nicht trainiert. Die meisten Modelle verwenden dieselbe Vorverarbeitung aus Abschnitt 5.1, damit die Ergebnisse später fair miteinander verglichen werden können. Dabei werden Textspalten mit TF-IDF verarbeitet, kategoriale Merkmale One-Hot-codiert und numerische Merkmale skaliert.

Der `DummyClassifier` dient als einfache Referenz. Er zeigt, welche Leistung ein Modell ohne echte Mustererkennung erreicht. Darauf aufbauend werden mehrere lineare Textmodelle definiert. Die `LinearSVC`-Modelle bilden starke klassische SVM-Benchmarks für hochdimensionale sparse TF-IDF-Merkmale. Dabei werden Klassengewichtung und Regularisierung variiert. Ein kleineres `C` bedeutet stärkere Regularisierung und kann bei sehr vielen TF-IDF-Merkmalen zu stabilerer Optimierung führen.

Zusätzlich wird `SGDClassifier(loss="hinge")` als schnelle SVM-nahe Alternative aufgenommen. Dieses Modell approximiert eine lineare SVM mit stochastischem Gradientenverfahren und kann bei großen sparse Matrizen schneller trainieren als `LinearSVC`. Der `SGDClassifier(loss="log_loss")` ergänzt dies als schnelle logistische Variante, weil er eine logistische Verlustfunktion mit stochastischem Gradientenverfahren optimiert.

Die reguläre logistische Regression bleibt als interpretierbare lineare Baseline enthalten. `ComplementNB` wird als sehr schnelle textbasierte Zusatzbaseline aufgenommen. Da dieses Modell besonders für sparse, nicht-negative Textmerkmale wie TF-IDF geeignet ist, wird es bewusst nur mit den Textspalten verwendet.

CatBoost wird nicht in die scikit-learn-`Pipeline` der linearen Modelle aufgenommen, weil es Text-, kategoriale und numerische Merkmale mit einer eigenen Datenstruktur (`Pool`) verarbeitet. Deshalb wird CatBoost im Baseline-Abschnitt separat, aber mit denselben Metriken und demselben Cross-Validation-Split evaluiert. Dadurch bleibt der Vergleich methodisch konsistent, während CatBoost seine eigene Verarbeitung gemischter Merkmalsarten nutzen kann.

5.4.1 Klassischen ML-Modelle

► Code

	Modell	Classifier	Verwendet_Preprocessing
0	Dummy Classifier	DummyClassifier	False
1	Linear SVC unweighted	LinearSVC	True
2	Linear SVC balanced	LinearSVC	True
3	SGD hinge balanced	SGDClassifier	True
4	SGD log loss balanced	SGDClassifier	True
5	Logistic Regression balanced	LogisticRegression	True

	Modell	Classifier	Verwendet_Preprocessing
6	Complement Naive Bayes text only	ComplementNB	True

5.5. Baseline Cross-Validation mit MLflow

In diesem Abschnitt werden die zuvor definierten Baseline-Modelle mit derselben Cross-Validation und denselben Metriken bewertet. Ziel ist nicht die finale Modelloptimierung, sondern ein fairer erster Vergleich der Modellklassen.

MLflow wird dabei als Experiment-Tracking-Werkzeug verwendet. Es speichert für jeden Modelllauf die wichtigsten Parameter, Metriken und Ergebnisdateien. Dadurch bleiben die Experimente reproduzierbar und später vergleichbar, auch wenn weitere Modelle oder Optuna-Tuning hinzukommen.

5.5.1. MLflow vorbereiten

Zuerst wird ein lokaler MLflow-Tracking-Ordner im Projektverzeichnis definiert. Alle Baseline-Ergebnisse werden unter demselben Experimentnamen gespeichert. So lassen sich die Läufe später über die MLflow UI vergleichen.

► Code

```
MLflow Tracking URI: file:///d:/toydev/schwarz-test/mlruns
MLflow Experiment: politikbereich_classifier
```

5.5.2. Hilfsfunktionen für Logging und Auswertung

Die folgenden Funktionen kapseln die wiederkehrenden Schritte. Für jedes Modell werden die Cross-Validation-Ergebnisse berechnet, Mittelwert und Standardabweichung der Metriken gebildet und anschließend in MLflow gespeichert. Zusätzlich wird die detaillierte Fold-Auswertung als CSV-Artefakt abgelegt.

► Code

5.5.3. Baseline-Modelle evaluieren

Nun werden alle definierten Baseline-Modelle mit derselben stratifizierten Cross-Validation bewertet. Falls für ein Modell bereits ein gespeichertes Ergebnis existiert, wird dieses wiederverwendet und das Modell wird nicht erneut trainiert. Neue Modelle werden berechnet und anschließend an die bestehenden Ergebnisdateien angehängt.

Zusätzlich werden `fit_time` und `score_time` gespeichert. Dadurch kann nicht nur die Modellgüte, sondern auch der Rechenaufwand der einzelnen Modellvarianten verglichen werden. Der Testdatensatz wird hier weiterhin nicht verwendet. Er bleibt für die finale Bewertung des ausgewählten Modells reserviert.

► Code

```
Verwende gespeichertes Ergebnis für: Dummy Classifier
Verwende gespeichertes Ergebnis für: Linear SVC unweighted
Verwende gespeichertes Ergebnis für: Linear SVC balanced
Verwende gespeichertes Ergebnis für: SGD hinge balanced
```

Verwende gespeichertes Ergebnis für: SGD log loss balanced
Verwende gespeichertes Ergebnis für: Logistic Regression balanced
Verwende gespeichertes Ergebnis für: Complement Naive Bayes text only

	Modell	CV_Folds	macro_f1_mean	macro_f1_std	weighted_f1_mean	weighted_f1_std	balanced_accuracy_mean
0	Logistic Regression balanced	4	0.8246	0.0060	0.9068	0.0037	0.8533
1	Linear SVC unweighted	4	0.8225	0.0167	0.9266	0.0014	0.7921
2	Linear SVC balanced	4	0.8164	0.0071	0.9138	0.0008	0.8432
3	SGD hinge balanced	4	0.8062	0.0209	0.9220	0.0037	0.8381
4	SGD log loss balanced	4	0.7793	0.0080	0.8817	0.0031	0.8145
5	Complement Naive Bayes text only	4	0.6306	0.0041	0.8539	0.0017	0.5931
6	Dummy Classifier	4	0.0103	0.0000	0.0653	0.0000	0.0312



Die Baseline-Ergebnisse zeigen, dass lineare TF-IDF-basierte Modelle bereits eine starke Vergleichsbasis liefern. Entscheidend ist dabei der Macro-F1-Score, weil die Zielvariable deutlich unausgewogen ist. Der Abstand zum **DummyClassifier** zeigt, dass die Modelle verwertbare Muster aus den Eingabemerkmale lernen und nicht nur die häufigste Klasse reproduzieren.

5.5.4. CatBoost-Baseline mit gemischten Merkmalen

CatBoost wird als zusätzliche Baseline für gemischte Merkmale verwendet. Im Unterschied zu den linearen TF-IDF-Modellen verarbeitet CatBoost Textspalten, kategoriale Spalten und numerische Spalten gemeinsam. Dadurch wird geprüft, ob ein nicht-lineares Boosting-Modell zusätzliche Muster aus der Kombination von **name**, **geber**, **anschrift**, **zweck**, **art**, **jahr** und **betrag** lernen kann.

Die Evaluation verwendet dieselben Folds und dieselben Metriken wie die übrigen Baseline-Modelle. Falls CatBoost in der Umgebung nicht installiert ist, wird der Schritt übersprungen und kann nach der Installation erneut ausgeführt werden.

► Code

Starte Cross-Validation für: CatBoost mixed features

5.5.5. Ergebnis speichern

Die zusammengefassten Baseline-Ergebnisse werden zusätzlich als CSV-Datei im Projekt gespeichert. Diese Datei kann später im Optuna-Notebook verwendet werden, um zu entscheiden, welche Modelle für das Tuning

priorisiert werden.

► Code

Gespeichert: d:\toydev\schwarz-test\data\processed\baseline_cv_results.csv

Gespeichert: d:\toydev\schwarz-test\data\processed\baseline_cv_fold_results.csv

5.6. Feature-Ablation und Repräsentationsexperimente

Die folgenden Experimente bauen auf den Beobachtungen aus der explorativen Datenanalyse auf. Besonders `name` und `anschrift` stehen stark mit der Empfänger-ID in Verbindung und können deshalb als indirekte Identifikationsmerkmale wirken. `zweck` beschreibt dagegen den inhaltlichen Förderzweck und ist damit die fachlich naheliegendste Informationsquelle für den Politikbereich. `geber` kann zusätzlich als administrativer Kontext wirken, während `art`, `jahr` und `betrag` strukturierte Zusatzinformationen darstellen.

Es werden zwei Experimenttypen unterschieden. Bei der **Feature-Ablation** werden einzelne Informationsquellen entfernt oder hinzugefügt, um ihren Beitrag zur Modellleistung zu prüfen. Bei den **Repräsentationsexperimenten** bleibt die Information grundsätzlich erhalten, wird aber anders vorverarbeitet, zum Beispiel `geber` als Textmerkmal oder als kategoriales Merkmal. Damit die Ergebnisse interpretierbar bleiben, wird das Modell festgehalten und nur die verwendeten Spalten bzw. deren Repräsentation werden variiert.

5.6.1. Experimentdesign

Die Feature-Ablationen prüfen systematisch den Beitrag einzelner Informationsquellen. Die Repräsentationsexperimente untersuchen dagegen, ob `geber` und `art` besser als Textmerkmale oder als kategoriale Merkmale verarbeitet werden sollten. Das ist besonders für `geber` relevant, weil die Spalte zwar nur wenige Ausprägungen enthält, die Namen der Verwaltungseinheiten aber selbst Begriffe wie Kultur, Wirtschaft oder Europa enthalten können.

► Code

	Experimenttyp	Beschreibung	Textspalten	Kategoriale Spalten	Numerische Spalten
Ablation					
Full model	Feature-Ablation	Alle vorgesehenen Merkmalsgruppen	name_standardised, geber_standardised, anschri...	art_standardised, jahr	betrag
Zweck only	Feature-Ablation	Nur inhaltlicher Förderzweck	zweck_standardised		
Zweck + geber text	Repräsentationsexperiment	Förderzweck plus Geber als Text	zweck_standardised, geber_standardised		
Zweck + geber category	Repräsentationsexperiment	Förderzweck plus Geber als Kategorie	zweck_standardised	geber_standardised	

	Experimenttyp	Beschreibung	Textspalten	Kategoriale Spalten	Numerische Spalten
Ablation					
Ohne Empfängeridentität	Feature-Ablation	Ohne Name und Anschrift	geber_standardised, zweck_standardised	art_standardised, jahr	betrag
Ohne Anschrift	Feature-Ablation	Alle Merkmale außer Anschrift	name_standardised, geber_standardised, zweck_s...	art_standardised, jahr	betrag
Ohne Name	Feature-Ablation	Alle Merkmale außer Name	geber_standardised, anschrift_standardised, zw...	art_standardised, jahr	betrag
Text only	Feature-Ablation	Nur Textspalten ohne strukturierte Zusatzmerkmale	name_standardised, geber_standardised, anschri...		
Text + categorical	Feature-Ablation	Textspalten plus art und jahr	name_standardised, geber_standardised, anschri...	art_standardised, jahr	
Full with geber category	Repräsentationsexperiment	Geber als Kategorie statt als Text	name_standardised, anschrift_standardised, zwe...	geber_standardised, art_standardised, jahr	betrag
Full with geber and art as text	Repräsentationsexperiment	Geber und art beide als Text	name_standardised, geber_standardised, art_sta...	jahr	betrag

5.6.2. Festes Modell für die Experimente auswählen

Für die Feature-Ablation und die Repräsentationsexperimente wird ein Modell festgehalten. Da diese Experimente viele Varianten trainieren, wird nicht automatisch das insgesamt beste Modell gewählt, sondern das beste Modell innerhalb der schnell trainierbaren Baseline-Modelle. Dadurch bleibt die Laufzeit kontrollierbar, während die Auswahl weiterhin datenbasiert über die Hauptmetrik `macro_f1` erfolgt.

Als schnelle Modelle gelten hier Modelle mit einer durchschnittlichen Trainingszeit pro Cross-Validation-Fold von höchstens `max_fast_fit_time_seconds`. Falls für einzelne Modelle noch keine Laufzeit gespeichert wurde, werden bevorzugt die explizit schnellen linearen SGD-Modelle und `ComplementNB` berücksichtigt.

► Code

Maximale durchschnittliche Trainingszeit: 5.0

Schnelle Modellkandidaten:

	Modell	macro_f1_mean	fit_time_mean_seconds
0	SGD hinge balanced	0.8062	2.51
1	SGD log loss balanced	0.7793	2.52

Modell	macro_f1_mean	fit_time_mean_seconds
2 Complement Naive Bayes text only	0.6306	2.10

Modell für Feature-Ablation und Repräsentationsexperimente: SGD hinge balanced
Classifier: SGDClassifier

5.6.3. Experiment-Pipeline erzeugen

Für jede Variante wird ein eigener `ColumnTransformer` erzeugt. Dadurch können dieselben Spalten je nach Testbedingung unterschiedlich verarbeitet werden, zum Beispiel `geber` als Text oder als kategoriales Merkmal.

► Code

5.6.4. Cross-Validation und MLflow ausführen

Die Ergebnisse werden wie die Baseline-Ergebnisse zwischengespeichert. Bereits vorhandene Varianten werden nicht erneut trainiert. Neue Varianten werden berechnet, an die bestehenden Ergebnisdateien angehängt und zusätzlich in MLflow dokumentiert.

► Code

Verwende gespeichertes Ergebnis für: Full model
Verwende gespeichertes Ergebnis für: Zweck only
Verwende gespeichertes Ergebnis für: Zweck + geber text
Verwende gespeichertes Ergebnis für: Zweck + geber category
Verwende gespeichertes Ergebnis für: Ohne Empfängeridentität
Verwende gespeichertes Ergebnis für: Ohne Anschrift
Verwende gespeichertes Ergebnis für: Ohne Name
Verwende gespeichertes Ergebnis für: Text only
Verwende gespeichertes Ergebnis für: Text + categorical
Verwende gespeichertes Ergebnis für: Full with geber category
Verwende gespeichertes Ergebnis für: Full with geber and art as text

	macro_f1_mean	macro_f1_std	weighted_f1_mean	weighted_f1_std	balanced_accuracy_mean	balanced_accuracy_std
Ablation						
Text + categorical	0.8170	0.0132	0.9229	0.0009	0.8548	0.0011
Full with geber category	0.8117	0.0147	0.9231	0.0023	0.8468	0.0101
Full model	0.8062	0.0209	0.9220	0.0037	0.8381	0.0202
Text only	0.8020	0.0080	0.9256	0.0016	0.8574	0.0101
Ohne Name	0.7965	0.0177	0.9169	0.0040	0.8356	0.0101

	macro_f1_mean	macro_f1_std	weighted_f1_mean	weighted_f1_std	balanced_accuracy_mean	balanced_accuracy_std
Ablation						
Full with geber and art as text	0.7954	0.0210	0.9228	0.0043	0.8482	0.0117
Ohne Anschrift	0.7918	0.0191	0.9210	0.0023	0.8296	0.0117
Ohne Empfängeridentität	0.7664	0.0192	0.8970	0.0038	0.8149	0.0117
Zweck + geber category	0.7525	0.0073	0.9054	0.0014	0.8188	0.0117
Zweck + geber text	0.7483	0.0135	0.9042	0.0019	0.8167	0.0117
Zweck only	0.5994	0.0128	0.8304	0.0020	0.7254	0.0117

11 rows × 30 columns

► Code

Bestes Szenario: Text + categorical
Macro-F1: 0.817

Trainingsspalte	
0	name_standardised
1	geber_standardised
2	anschrift_standardised
3	zweck_standardised
4	art_standardised
5	jahr

Bestes Szenario: Text + categorical

Anzahl verwendeter Spalten: 6

Anzahl nicht verwendeter Spalten: 1

Nicht verwendete Trainingsspalte	
0	betrag

5.6.5. Kontrollvergleich mit Logistischer Regression (Aufgrund der Zeitaufwand noch nicht ausgeführt)

Zur Prüfung der Ablationsergebnisse wird zusätzlich ein Kontrollvergleich durchgeführt. Dabei bleibt das Modell konstant: Es wird immer **Logistic Regression balanced** verwendet. Verändert werden nur die verwendeten

Eingabespalten. Verglichen wird das vollständige Ausgangsszenario mit dem besten Szenario aus der Feature-Ablation bzw. den Repräsentationsexperimenten.

Damit lässt sich prüfen, ob die beobachtete Verbesserung tatsächlich auf die veränderte Datengrundlage zurückzuführen ist und nicht nur auf den zuvor verwendeten schnelleren Modelltyp.

► Code

KeyboardInterrupt Traceback (most recent call last)

Cell In[261], line 87

```
67 scenario_preprocessor = build_ablation_preprocessor(
68     scenario_config["text_features"],
69     scenario_config["categorical_features"],
70     scenario_config["numeric_features"],
71 )
73 scenario_model = Pipeline(
74     steps=[
75         (
76             ...
77             memory=sklearn_memory,
78             )
79 )
---> 87 scenario_cv_results = cross_validate(
88     scenario_model,
89     X_train[scenario_features].copy(),
90     y_train,
91     cv=cv,
92     scoring=scoring,
93     n_jobs=1,
94     return_train_score=True,
95 )
97 scenario_summary = create_cv_summary_row(
98     scenario_label,
99     scenario_cv_results,
100 )
102 model_path = fit_and_save_model(
103     scenario_model,
104     f"{logreg_control_model_name}__{scenario_label}",
105     ...
107     subdirectory="control_comparison",
108 )
```

File d:\toydev\schwarz-test\.venv\lib\site-packages\sklearn\utils_param_validation.py:218, in vali

```
212 try:
213     with config_context(
214         skip_parameter_validation=(
215             prefer_skip_nested_validation or global_skip_validation
216         )
217     ):
--> 218     return func(*args, **kwargs)
219 except InvalidParameterError as e:
```

```

220     # When the function is just a wrapper around an estimator, we allow
221     # the function to delegate validation to the estimator, but we replace
222     # the name of the estimator by the name of the function in the error
223     # message to avoid confusion.
224     msg = re.sub(
225         r"parameter of \w+ must be",
226         f"parameter of {func.__qualname__} must be",
227         str(e),
228     )

```

File `d:\toydev\schwarz-test\.venv\lib\site-packages\sklearn\model_selection\_validation.py:399`, in

```

396 # We clone the estimator to make sure that all the folds are
397 # independent, and that it is pickle-able.
398 parallel = Parallel(n_jobs=n_jobs, verbose=verbose, pre_dispatch=pre_dispatch)
--> 399 results = parallel(
400     delayed(_fit_and_score)(
401         clone(estimator),
402         X,
403         y,
404         scorer=scorers,
405         train=train,
406         test=test,
407         verbose=verbose,
408         parameters=None,
409         fit_params=routed_params.estimator.fit,
410         score_params=routed_params.scorer.score,
411         return_train_score=return_train_score,
412         return_times=True,
413         return_estimator=return_estimator,
414         error_score=error_score,
415     )
416     for train, test in indices
417 )
419 _warn_or_raise_about_fit_failures(results, error_score)
421 # For callable scoring, the return type is only know after calling. If the
422 # return type is a dictionary, the error scores can now be inserted with
423 # the correct key.

```

File `d:\toydev\schwarz-test\.venv\lib\site-packages\sklearn\utils\parallel.py:82`, in `Parallel.__call`

```

73 warning_filters = warnings.filters
74 iterable_with_config_and_warning_filters = (
75     (
76         _with_config_and_warning_filters(delayed_func, config, warning_filters),
77     )
78     for delayed_func, args, kwargs in iterable
79 )
--> 82 return super().__call__(iterable_with_config_and_warning_filters)

```

File `d:\toydev\schwarz-test\.venv\lib\site-packages\joblib\parallel.py:1986`, in `Parallel.__call__(s`

```

1984     output = self._get_sequential_output(iterable)
1985     next(output)

```

```
-> 1986     return output if self.return_generator else list(output)
1988 # Let's create an ID that uniquely identifies the current call. If the
1989 # call is interrupted early and that the same instance is immediately
1990 # reused, this id will be used to prevent workers that were
1991 # concurrently finalizing a task from the previous call to run the
1992 # callback.
1993 with self._lock:
```

```
File d:\toydev\schwarz-test\.venv\lib\site-packages\joblib\parallel.py:1914, in Parallel._get_seque
1912 self.n_dispatched_batches += 1
1913 self.n_dispatched_tasks += 1
-> 1914 res = func(*args, **kwargs)
1915 self.n_completed_tasks += 1
1916 self.print_progress()
```

```
File d:\toydev\schwarz-test\.venv\lib\site-packages\sklearn\utils\parallel.py:147, in _FuncWrapper.
145 with config_context(**config), warnings.catch_warnings():
146     warnings.filters = warning_filters
--> 147     return self.function(*args, **kwargs)
```

```
File d:\toydev\schwarz-test\.venv\lib\site-packages\sklearn\model_selection\_validation.py:859, in
857     estimator.fit(X_train, **fit_params)
858     else:
--> 859     estimator.fit(X_train, y_train, **fit_params)
861 except Exception:
862     # Note fit time as time until error
863     fit_time = time.time() - start_time
```

```
File d:\toydev\schwarz-test\.venv\lib\site-packages\sklearn\base.py:1365, in _fit_context.<locals>.
1358     estimator._validate_params()
1360 with config_context(
1361     skip_parameter_validation=(
1362         prefer_skip_nested_validation or global_skip_validation
1363     )
1364 ):
-> 1365     return fit_method(estimator, *args, **kwargs)
```

```
File d:\toydev\schwarz-test\.venv\lib\site-packages\sklearn\pipeline.py:663, in Pipeline.fit(self,
657     if self._final_estimator != "passthrough":
658         last_step_params = self._get_metadata_for_step(
659             step_idx=len(self) - 1,
660             step_params=routed_params[self.steps[-1][0]],
661             all_params=params,
662         )
--> 663     self._final_estimator.fit(Xt, y, **last_step_params["fit"])
665 return self
```

```
File d:\toydev\schwarz-test\.venv\lib\site-packages\sklearn\base.py:1365, in _fit_context.<locals>.
1358     estimator._validate_params()
1360 with config_context(
1361     skip_parameter_validation=(
```

```

1362         prefer_skip_nested_validation or global_skip_validation
1363     )
1364 ):
-> 1365     return fit_method(estimator, *args, **kwargs)

```

File `d:\toydev\schwarz-test\.venv\lib\site-packages\sklearn\linear_model\_logistic.py:1384`, in `Logit`

```

1381 else:
1382     n_threads = 1
-> 1384 fold_coefs_ = Parallel(n_jobs=self.n_jobs, verbose=self.verbose, prefer=prefer)(
1385     path_func(
1386         X,
1387         y,
1388         pos_class=class_,
1389         Cs=[C_],
1390         l1_ratio=self.l1_ratio,
1391         fit_intercept=self.fit_intercept,
1392         tol=self.tol,
1393         verbose=self.verbose,
1394         solver=solver,
1395         multi_class=multi_class,
1396         max_iter=self.max_iter,
1397         class_weight=self.class_weight,
1398         check_input=False,
1399         random_state=self.random_state,
1400         coef=warm_start_coef_,
1401         penalty=penalty,
1402         max_squared_sum=max_squared_sum,
1403         sample_weight=sample_weight,
1404         n_threads=n_threads,
1405     )
1406     for class_, warm_start_coef_ in zip(classes_, warm_start_coef)
1407 )
1409 fold_coefs_, _, n_iter_ = zip(*fold_coefs_)
1410 self.n_iter_ = np.asarray(n_iter_, dtype=np.int32)[: , 0]

```

File `d:\toydev\schwarz-test\.venv\lib\site-packages\sklearn\utils\parallel.py:82`, in `Parallel.__call`

```

73 warning_filters = warnings.filters
74 iterable_with_config_and_warning_filters = (
75     (
76         _with_config_and_warning_filters(delayed_func, config, warning_filters),
77     )
78     for delayed_func, args, kwargs in iterable
79 )
---> 82 return super().__call__(iterable_with_config_and_warning_filters)

```

File `d:\toydev\schwarz-test\.venv\lib\site-packages\joblib\parallel.py:2072`, in `Parallel.__call__(self, iterable, **kwargs)`

```

2066 # The first item from the output is blank, but it makes the interpreter
2067 # progress until it enters the Try/Except block of the generator and
2068 # reaches the first `yield` statement. This starts the asynchronous
2069 # dispatch of the tasks to the workers.
2070 next(output)

```

```
-> 2072 return output if self.return_generator else list(output)
```

File d:\toydev\schwarz-test\.venv\lib\site-packages\joblib\parallel.py:1682, in Parallel._get_outpu

```
1679     yield
1681     with self._backend.retrieval_context():
-> 1682         yield from self._retrieve()
1684 except GeneratorExit:
1685     # The generator has been garbage collected before being fully
1686     # consumed. This aborts the remaining tasks if possible and warn
1687     # the user if necessary.
1688     self._exception = True
```

File d:\toydev\schwarz-test\.venv\lib\site-packages\joblib\parallel.py:1800, in Parallel._retrieve(

```
1789 if self.return_ordered:
1790     # Case ordered: wait for completion (or error) of the next job
1791     # that have been dispatched and not retrieved yet. If no job
1792     (...)
1795     # control only have to be done on the amount of time the next
1796     # dispatched job is pending.
1797     if (nb_jobs == 0) or (
1798         self.__jobs[0].get_status(timeout=self.timeout) == TASK_PENDING
1799     ):
-> 1800         time.sleep(0.01)
1801         continue
1803 elif nb_jobs == 0:
1804     # Case unordered: jobs are added to the list of jobs to
1805     # retrieve `self.__jobs` only once completed or in error, which
1806     (...)
1811     # timeouts before any other dispatched job has completed and
1812     # been added to `self.__jobs` to be retrieved.
```

KeyboardInterrupt:

5.6.6. Ergebnisse speichern

Die Ergebnisse werden gespeichert, damit spätere Notebook-Läufe bereits berechnete Varianten wiederverwenden können. Die besten Merkmals- und Repräsentationsentscheidungen dienen anschließend als Grundlage für die Hyperparameteroptimierung mit Optuna.

► Code

Gespeichert: d:\toydev\schwarz-test\data\processed\ablation_cv_results.csv

Gespeichert: d:\toydev\schwarz-test\data\processed\ablation_cv_fold_results.csv

5.7. Erweiterte NLP- und Deep-Learning-Modelle

Neben den klassischen scikit-learn-Modellen werden zwei weitere Modellfamilien vorbereitet: `fastText` und ein deutschsprachiges bzw. multilingual einsetzbares BERT-Modell. Beide Modelle benötigen eine andere

Datenrepräsentation als die bisherige `ColumnTransformer`-Pipeline. Deshalb werden sie hier nicht direkt in die bestehende Cross-Validation integriert, sondern als optionale Erweiterung vorbereitet.

`fastText` erwartet eine Textdatei mit Labels im Format `__label__Klasse Text`. BERT arbeitet dagegen mit einem Tokenizer, der den zusammengeführten Text in Token-IDs übersetzt. Die folgenden Zellen definieren daher die jeweils passende Eingaberepräsentation. Die eigentliche Modellschätzung ist bewusst deaktiviert und kann bei ausreichender Rechenzeit eingeschaltet werden.

5.7.1. Gemeinsame Textrepräsentation für NLP-Modelle

Für beide Modelle werden die relevanten Modellspalten zu einem einzigen Textfeld zusammengeführt. Dabei bleiben die Spaltennamen als Feldmarker erhalten. Dadurch kann das Modell unterscheiden, ob ein Begriff aus dem Namen, dem Zweck, der Anschrift oder einer kategorialen Angabe stammt.

Als Ausgangspunkt wird das beste Szenario aus den vorherigen Repräsentationsexperimenten verwendet. Falls dieses Objekt bei einem isolierten Notebook-Lauf noch nicht existiert, wird eine robuste Standardauswahl aus Text- und kategorialen Spalten verwendet.

► Code

Verwendete Spalten für NLP-Modelle:

Trainingsspalte	
0	name_standardised
1	geber_standardised
2	anschrift_standardised
3	zweck_standardised
4	art_standardised
5	jahr

model_text	
id	
33655	name: cashmere radio e. v. geber: senatsverwal...
105910	name: merantix labs gmbh geber: senatsverwalu...
67465	name: georg kolbe-stiftung geber: senatsverwal...

5.7.2. fastText vorbereiten

`fastText` ist für Textklassifikation sehr effizient, weil es Wort- und n-Gramm-Informationen direkt aus einer einfachen Textdatei lernt. Es passt jedoch nicht direkt in die bestehende scikit-learn-Pipeline. Deshalb wird hier zuerst eine fastText-kompatible Trainingsdatei erzeugt.

Die Trainingszelle ist standardmäßig deaktiviert. Wenn `fasttext` installiert ist und das Modell tatsächlich trainiert werden soll, kann `train_fasttext_model` auf `True` gesetzt werden.

► Code

fastText-Trainingsdatei gespeichert: d:\toydev\schwarz-test\data\processed\fasttext_train.txt

	label	text
id		
33655	__label__Kultur	name: cashmere radio e. v. geber: senatsverwal...
105910	__label__Wirtschaft	name: merantix labs gmbh geber: senatsverwal...
67465	__label__Kultur	name: georg kolbe-stiftung geber: senatsverwal...

fastText-Training ist deaktiviert. Setze `train_fasttext_model = True`, um es auszuführen.

5.7.3. German BERT / multilingual BERT vorbereiten

BERT-Modelle verwenden keinen TF-IDF-Vektorizer, sondern einen Tokenizer. Dieser zerlegt den zusammengeführten Text in Token-IDs, Attention Masks und weitere Eingaben für ein Transformer-Modell. Dadurch kann semantischer Kontext besser berücksichtigt werden, allerdings ist das Fine-Tuning deutlich rechenintensiver als die klassischen Baselines.

Hier wird nur die benötigte Daten- und Encoderstruktur vorbereitet. Das eigentliche Fine-Tuning bleibt deaktiviert, damit der Baseline-Notebook-Lauf weiterhin leicht reproduzierbar bleibt.

► Code

BERT-Labelmapping gespeichert: d:\toydev\schwarz-test\data\processed\bert_label_mapping.csv

	label_id	politikbereich
0	0	Antidiskriminierung
1	1	Arbeit
2	2	Bauen, Wohnen
3	3	Berlin-Image
4	4	Bildung

BERT-Tokenisierung ist deaktiviert. Setze `prepare_bert_tokenizer = True`, um sie auszuführen.

5.7.4. fastText fitten, auswerten und speichern

Für fastText wird aus den Trainingsdaten ein kleiner interner Validierungssplit erzeugt. Dadurch kann das Modell direkt beurteilt werden, ohne den finalen Testdatensatz zu verwenden. Das Modell wird anschließend als `.bin`-Datei gespeichert. Zusätzlich werden Labelmapping, Summary und klassenbezogener Report abgelegt.

► Code

fastText-Modell gespeichert: d:\toydev\schwarz-test\models\fasttext\fasttext_politikbereich.bin
fastText-Summary gespeichert: d:\toydev\schwarz-test\models\fasttext\fasttext_summary.csv

	model	validation_rows	accuracy	macro_f1	weighted_f1	balanced_accuracy	training_time_seconds	model_path
0	fastText	4592	0.938371	0.743547	0.93715	0.717156	58.59	d:\toydev\schw test\models\fast



	precision	recall	f1-score	support
Antidiskriminierung	0.848485	0.835821	0.842105	67.0
Arbeit	0.968799	0.993600	0.981043	625.0
Bauen, Wohnen	0.833333	0.625000	0.714286	8.0
Bildung	0.966790	0.970370	0.968577	270.0
Bürgerschaftliches Engagement, Bürgerbeteiligung	0.743056	0.764286	0.753521	140.0
Denkmalschutz	1.000000	0.761905	0.864865	21.0
Europa	0.750000	0.333333	0.461538	9.0
Familie	0.968421	0.910891	0.938776	101.0
Finanzen	0.000000	0.000000	0.000000	1.0
Forschung	0.920000	1.000000	0.958333	23.0

fastText dient als effizienter NLP-Vergleich zu den klassischen TF-IDF-Modellen. Die Ergebnisse werden auf einem internen Validierungssplit berechnet und sind daher nicht direkt mit der finalen Testevaluation gleichzusetzen. Sie zeigen aber, ob eine kompakte n-Gramm-basierte Textrepräsentation als zusätzliche Modellfamilie konkurrenzfähig sein kann.

5.7.5. BERT fitten, auswerten und speichern

Für BERT wird ebenfalls nur ein interner Validierungssplit aus den Trainingsdaten verwendet. Das Fine-Tuning ist deutlich rechenintensiver als die klassischen Modelle und sollte vorzugsweise in Colab mit GPU ausgeführt werden. Das trainierte Modell, der Tokenizer, das Labelmapping und die Validierungsergebnisse werden gespeichert.

► Code